

System Monitoring and Process Management Notes

I. Process Tracking with `top`

Overview

The `top` command provides real-time system utilization monitoring by processes.

Sample `vmstat` Output

```
top - 18:06:26 up 6 days, 4:07, 2 users, load average: 0.92, 0.62, 0.59
Tasks: 389 total, 1 running, 387 sleeping, 0 stopped, 1 zombie
%Cpu(s): 1.8 us, 0.4 sy, 0.0 ni, 97.6 id, 0.1 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem: 32870888 total, 27467976 used, 5402912 free, 518808 buffers
KiB Swap: 33480700 total, 39892 used, 33440808 free. 19454152 cached Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM     TIME+ COMMAND
 6675 patty    20   0 1731472 520960 30876 S   8.3   1.6 160:24.79 chrome
 6926 patty    20   0  935888 163456 25576 S   4.3   0.5   5:28.13 chrome
```

Header Information Breakdown

Line 1 - System Overview:

- Current time: 18:06:26
- Uptime: 6 days, 4 hours, 7 minutes
- Users logged in: 2
- Load average: 0.92, 0.62, 0.59 (1, 5, 15 minute intervals)

Line 2 - Task Status:

- Total tasks: 389
- Running: 1, Sleeping: 387, Stopped: 0, Zombie: 1

Line 3 - CPU Information:

- `us` (user): 1.8% - CPU time for user processes (not niced)

- **sy** (system): 0.4% - CPU time for kernel processes
- **ni** (nice): 0.0% - CPU time for niced processes
- **id** (idle): 97.6% - CPU idle time
- **wa** (I/O wait): 0.1% - CPU time waiting for I/O
- **hi** (hardware interrupts): 0.0% - CPU time serving hardware interrupts
- **si** (software interrupts): 0.0% - CPU time serving software interrupts
- **st** (steal time): 0.0% - CPU time stolen by virtual machines

Lines 4-5 - Memory Information:

- Physical memory usage
- Swap usage and cache information

Process List Columns

- **PID**: Process ID
- **USER**: Process owner
- **PR**: Priority
- **NI**: Nice value
- **VIRT**: Virtual memory used
- **RES**: Physical memory used
- **SHR**: Shared memory
- **S**: Status (S=sleep, R=running, Z=zombie, D=uninterruptible, T=stopped)
- **%CPU**: CPU percentage used
- **%MEM**: RAM percentage used
- **TIME+**: Total activity time
- **COMMAND**: Process name

Top Command Usage Examples

```
# Monitor specific process
$ top -p 1
```

2. File and Process Tracking

lsuf (List Open Files)

Lists all open files and their associated processes. Remember: everything is a file in Linux (disks, pipes, network sockets, devices, etc.).

Example:

```
pete@icebox:~$ lsof .
COMMAND    PID  USER  FD   TYPE DEVICE SIZE/OFF  NODE NAME
lxsession  1491 pete   cwd   DIR    8,6    4096   131 .
update-no  1796 pete   cwd   DIR    8,6    4096   131 .
nm-applet  1804 pete   cwd   DIR    8,6    4096   131 .
indicator  1809 pete   cwd   DIR    8,6    4096   131 .
xterm      2205 pete   cwd   DIR    8,6    4096   131 .
bash       2207 pete   cwd   DIR    8,6    4096   131 .
lsof       5914 pete   cwd   DIR    8,6    4096   131 .
lsof       5915 pete   cwd   DIR    8,6    4096   131 .
```

Use Case: Finding which processes are using a USB device that won't unmount ("Device or Resource Busy" error).

fuser (File User)

Shows information about processes using a specific file or directory.

Example:

```
pete@icebox:~$ fuser -v .
                USER      PID ACCESS COMMAND
/home/pete:    pete  1491 ..c.. lxsession
               pete  1796 ..c.. update-notifier
               pete  1804 ..c.. nm-applet
               pete  1809 ..c.. indicator-power
               pete  2205 ..c.. xterm
               pete  2207 ..c.. bash
```

3. Process Threads

Key Concepts

- **Single-threaded:** Process has one thread
- **Multi-threaded:** Process has multiple threads
- **Threads:** Lightweight processes that share system resources
- All processes have at least one thread

Real-world Example

- LibreOffice Writer (separate process) and Chrome (separate process)
- Within Writer: editing text and auto-saving run as separate threads

Viewing Threads

```
pete@icebox:~$ ps m
  PID TTY          STAT       TIME COMMAND
 2207 pts/2    -        0:01 bash
     -  -        Ss         0:01 -
 5252 pts/2    -        0:00 ps m
     -  -        R+         0:00 -
```

Note: PIDs show processes, entries with `--` show their threads (both examples above are single-threaded).

4. CPU Monitoring with `uptime`

Command and Output

```
pete@icebox:~$ uptime
17:23:35 up 1 day, 5:59, 2 users, load average: 0.00, 0.02, 0.05
```

Load Average Explanation

Load averages represent CPU load over 1, 5, and 15-minute intervals.

Traffic Analogy:

- Single-core CPU = single traffic lane
- Load of 1.0 = 100% utilization (rush hour traffic)
- Load of 2.0 = 200% utilization (traffic backing up)
- Load of 0.5 = 50% utilization (half capacity)

Multi-core Considerations:

- Quad-core processor with load average of 1.0 = 25% total CPU usage
- Check core count: `cat /proc/cpuinfo`

5. I/O Monitoring with `iostat`

`iostat` Example Output

```
pete@icebox:~$ iostat
Linux 3.13.0-39-lowlatency (icebox)      01/28/2016      _i686_   (1 CPU)
```

avg-cpu:	%user	%nice	%system	%iowait	%steal	%idle
	0.13	0.03	0.50	0.01	0.00	99.33

Device:	tps	kB_read/s	kB_wrtn/s	kB_read	kB_wrtn
sda	0.17	3.49	1.92	385106	212417

CPU Information

- **%user**: CPU utilization at user level (applications)
- **%nice**: CPU utilization at user level with nice priority
- **%system**: CPU utilization at system level (kernel)
- **%iowait**: CPU idle time waiting for disk I/O
- **%steal**: Time spent in involuntary wait by virtual CPU
- **%idle**: CPU idle time without outstanding disk I/O

Disk Utilization

- **tps**: Transfers per second to device
- **kB_read/s**: Data read from device (KB/sec)
- **kB_wrtn/s**: Data written to device (KB/sec)
- **kB_read**: Total kilobytes read
- **kB_wrtn**: Total kilobytes written

6. Memory Monitoring with `vmstat`

Example Output

```
pete@icebox:~$ vmstat
procs -----memory----- ---swap-- -----io----- -system-- -----cpu-
-----
r  b   swpd   free   buff  cache   si   so    bi    bo    in   cs  us  sy  id
wa st
1  0       0 396528 38816 384036    0    0    4    2   38   79   0   0 99
0  0
```

Field Breakdown

procs:

- **r**: Number of processes waiting for run time
- **b**: Number of processes in uninterruptible sleep

memory:

- **swpd**: Virtual memory used
- **free**: Free memory
- **buff**: Memory used as buffers
- **cache**: Memory used as cache

swap:

- **si**: Memory swapped in from disk
- **so**: Memory swapped out to disk

io:

- **bi**: Blocks received from block device
- **bo**: Blocks sent to block device

system:

- **in**: Interrupts per second
- **cs**: Context switches per second

cpu:

- **us**: Time spent in user time
- **sy**: Time spent in kernel time
- **id**: Time spent idle
- **wa**: Time spent waiting for I/O

7. Continuous Monitoring with **sar**

Installation

```
sudo apt install sysstat
```

Configuration

Enable data collection by modifying `/etc/default/sysstat` (usually auto-enabled).

Usage Examples

```
# View system queue details from start of day
$ sudo sar -q
```

```
# View memory usage details from start of day
$ sudo sar -r

# View CPU usage details
$ sudo sar -P

# View specific day (XX = day number)
$ sar -q /var/log/sysstat/sa02
```

Purpose: Historical analysis for machines experiencing issues when unattended.

8. Cron Jobs for Task Scheduling

Cron Job Format

```
30 08 * * * /home/pete/scripts/change_wallpaper
```

Field Structure (left to right)

1. **Minute** (0-59)
2. **Hour** (0-23)
3. **Day of month** (1-31)
4. **Month** (1-12)
5. **Day of week** (0-7, where 0 and 7 = Sunday)

Asterisk (*): Matches every value for that field

Example Breakdown

The example above runs at 8:30 AM every day:

- Minute: 30
- Hour: 08
- Day of month: * (every day)
- Month: * (every month)
- Day of week: * (every day of week)

Creating Cron Jobs

```
crontab -e
```

Real-world Example: Automatically change desktop wallpaper every morning at 8:30 AM using a script located at `/home/pete/scripts/change_wallpaper`.